

Solar Local Stack

The orchestration software + local telemetry abstraction layer — the system under design. Replaces dependence on Hoymiles S-Miles Cloud. Implements dynamic excess-solar diversion, auto-discharge at night, and outage coordination with the Battery System.

Version **0.1**

Dictionary last updated **2026-05-22**

Generated **2026-05-23**

Validation:  **valid**

Modernization: **ADRs: 2 · Budgets/TPMs: 2/3 · SLOs: 2 · Bounded contexts: 2**

Contents

Stage 1 — Context Diagram	32
---------------------------------	----

Stage 1 — Context Diagram

6 terminator(s)

Solar Local Stack — Context Diagram v0 (Mermaid)

Status: draft for review. Six uncertainty items pending Kevin's call (see PLAN.md > Open Questions).

Diagram

```
graph LR
    HM[Solar Inverters]
    CM[Net Power Meter]
    VX[Battery System]
    GRID[Utility Grid]
    USR[Owner]
    SMC([S-Miles Cloud<br/><i>optional</i>])

    SYS(("Solar Local Stack"))

    HM -- "F1: per-channel telemetry" --> SYS
    SYS -- "F2: power-limit setpoints" --> HM
    CM -- "F3: net grid power, V/I/PF" --> SYS
    VX -- "F4: SoC, mode, AC-in" --> SYS
    SYS -- "F5: max-charge, grid setpoint" --> VX
    USR -- "F6: config, override" --> SYS
    SYS -- "F7: dashboards, alerts" --> USR
    SYS -. "F8: optional telemetry forward" .-> SMC

    GRID --- HM
    GRID --- VX

    classDef system fill:#4a90e2,stroke:#2a70c2,color:#fff,font-weight:bold;
    classDef terminator fill:#fafafa,stroke:#444;
    classDef optional fill:#fafafa,stroke:#888,stroke-dasharray:5 5,color:#666;
    classDef grid fill:#fef0ef,stroke:#e74c3c;
    class SYS system;
    class HM,CM,VX,USR terminator;
    class SMC optional;
    class GRID grid;
```

Flow specification (the formal artifact)

ID	Source	Target	Name	Kind	Medium	Notes
F1	Solar Inverters	System	per-channel power / V / I / temp / fault state	data	Sub-1G RF → DTU-Pro-S <i>or</i> OpenDTU	~30 s native; near-realtime via OpenDTU
F2	System	Solar Inverters	per-channel power-limit setpoint	control	OpenDTU only	Conditional on DTU architectural choice
F3		System		data		

ID	Source	Target	Name	Kind	Medium	Notes
	Net Power Meter		net grid power, per-phase V/I/PF, direction		RS485 Modbus RTU	<i>primary</i> sensing input for diversion loop
F4	Battery System	System	battery SoC, charging state, AC-in power, system mode	data	MQTT / Modbus TCP	inbound side of bidirectional channel
F5	System	Battery System	max-charge current, grid setpoint, ESS params	control	MQTT publish	<i>primary</i> actuating output for diversion loop
F6	Owner	System	config, manual override, enable/disable	data + control	UI	mostly config; some are control events
F7	System	Owner	dashboard views, alerts, status	data	UI	reverse of F6
F8	System	S-Miles Cloud	optional telemetry forward	data	HTTPS	off by default

Open uncertainties (resolve next)

1. DTU choice — official DTU-Pro-S only, OpenDTU/AhoyDTU only, or both as variants? Affects F2.
2. PG&E Utility Meter as a distinct terminator from PG&E Grid, or rolled together?
3. EV charger / future loads — model now as placeholder, or defer?
4. Weather forecast service — in or out for the b+d cut?
5. S-Miles Cloud (F8) — keep dashed/optional, or drop entirely?
6. Outage handling — does the system *observe*, *participate*, or *stay out of the way*?

Stage 2 — Level-1 DFD

6 process(es)

Solar Local Stack — Level-1 DFD (Mermaid)

First decomposition of `sys_root` into 5 + 1 (optional) internal processes plus the `System State` data store. Locked 2026-05-22. Names from [../dictionary.yaml](https://github.com/mermaidjs/mermaid/blob/master/docs/diagrams/dfd/dfd-notation.md).

Diagram

```
graph LR
    %% External terminators
    HM[Solar Inverters]
    CM[Net Power Meter]
    VX[Battery System]
    GRID[Utility Grid]
    USR[Owner]
    SMC([S-Miles Cloud<br/><i>optional</i>])

    %% Internal processes
    P1((Acquire<br/>Telemetry))
    P2((Energy<br/>Manager))
    P3((Dispatch<br/>Commands))
    P4((Serve UI))
    P5((Handle<br/>Input))
    P6((Cloud Forward<br/><i>optional</i>))

    %% Data store
    DS[(System State)]

    %% Boundary flows F1-F8
    HM -- "F1: telemetry" --> P1
    P3 -- "F2: power-limit" --> HM
    CM -- "F3: net grid power" --> P1
    VX -- "F4: SoC, mode" --> P1
    P3 -- "F5: charge / setpoint" --> VX
    USR -- "F6: config, override" --> P5
    P4 -- "F7: dashboards" --> USR
    P6 -. "F8: telemetry fwd" .-> SMC

    %% Physical AC power
    GRID --- HM
    GRID --- VX

    %% Internal flows (event-driven per Decision 3)
    P1 -- "writes" --> DS
    DS -- "reads" --> P2
    DS -- "reads" --> P4
    DS -. "reads" .-> P6
    P2 -- "cmd_setpoint" --> P3
    P2 -- "cmd_inverter_limit" --> P3
    P2 -- "event_alert" --> P4
    P5 -- "event_override" --> P2
    P5 -. "event_config" .-> P1
```

```

%% Styling
classDef proc          fill:#cfe5ff,stroke:#2a70c2,color:#000;
classDef brain         fill:#7fbff5,stroke:#1f5a99,color:#000,font-weight:bold;
classDef optional      fill:#e6f0ff,stroke:#888,stroke-dasharray:3 3;
classDef terminator    fill:#fafafa,stroke:#444;
classDef store         fill:#fff5cc,stroke:#b89800;
classDef grid          fill:#fef0ef,stroke:#e74c3c;
classDef termopt       fill:#fafafa,stroke:#888,stroke-dasharray:5 5,color:#666;
class P1,P3,P4,P5 proc;
class P2 brain;
class P6 optional;
class DS store;
class HM,CM,VX,USR terminator;
class GRID grid;
class SMC termopt;

```

Balancing check

Every level-0 boundary flow appears at the level-1 boundary as well (HP balancing rule):

Level-0 flow	Level-1 source/target
F1 (in)	Solar Inverters → Acquire Telemetry
F2 (out)	Dispatch Commands → Solar Inverters
F3 (in)	Net Power Meter → Acquire Telemetry
F4 (in)	Battery System → Acquire Telemetry
F5 (out)	Dispatch Commands → Battery System
F6 (in)	Owner → Handle Input
F7 (out)	Serve UI → Owner
F8 (out)	Cloud Forward → S-Miles Cloud (<i>optional</i>)

✓ All 8 boundary flows preserved; no flows appear or disappear at the boundary.

Internal flows summary

Per Decision 3 (event-driven) and Decision 4 (explicit data store):

From	To	Carries	Kind
Acquire Telemetry	System State	normalized state	data (write)
System State	Energy Manager	system_state	data (read)
System State	Serve UI	system_state	data (read)
System State	Cloud Forward	system_state	data (read, optional)
Energy Manager	Dispatch Commands	cmd_setpoint	control
Energy Manager	Dispatch Commands	cmd_inverter_limit	control

From	To	Carries	Kind
Energy Manager	Serve UI	event_alert	control
Handle Input	Energy Manager	event_override	control
Handle Input	Acquire Telemetry	event_config	control (subtle)

Next: Stage 3

The **Energy Manager** bubble is state-rich — diversion loop, night-discharge, outage handling all live in its CSPEC. Stage 3 formalizes that as a state-transition diagram.

Stage 3 — CSPECs

1 CSPEC(s)

Energy Manager — CSPEC (Locked)

The [Control Specification](#) for the [Energy Manager](#) bubble. Hierarchical state machine — 4 top-level modes + sub-states. **Locked 2026-05-22** via [proposal.md](#) (decomposition decisions) + [naming-review.md](#) (state names).

State diagram

```
stateDiagram-v2
    direction LR

    [*] --> Initializing

    Initializing --> GridTie : telemetry healthy<br/>Victron mode = grid-tie
    Initializing --> Fault : startup failure

    state GridTie {
        direction LR
        [*] --> Idle
        Idle --> Diverting : surplus detected<br/>(net_grid_power < -threshold)
        Diverting --> Idle : balance reached<br/>( |net_grid_power| < deadband)
        Diverting --> Holding : battery SoC ≥ max
        Holding --> Diverting : surplus + battery headroom
        Idle --> Discharging : net import + night<br/>(time-of-use rule)
        Discharging --> Idle : battery SoC ≤ min<br/>OR sun returns
    }

    GridTie --> Island : Victron mode = island<br/>(grid lost)
    Island --> GridTie : Victron mode = grid-tie<br/>(grid restored)

    state Island {
        direction LR
        [*] --> BatteryDischarge
        BatteryDischarge --> SolarAssist : sun present<br/>+ Victron AC-couples
        SolarAssist --> BatteryDischarge : sun gone<br/>OR battery saturating
    }

    GridTie --> Fault : fault event
    Island --> Fault : fault event

    state Fault {
        direction LR
        [*] --> TelemetryFault
        TelemetryFault
        ActuatorFault
        PolicyFault
    }

    Fault --> GridTie : fault cleared<br/>+ grid present
    Fault --> Island : fault cleared<br/>+ grid absent
```


Transition table

(Event names below are *working* phrases — formalized into stable IDs in the next sub-stage's [event glossary](#), once we build it.)

From	To	Event (trigger)	Action (side effect)
Initializing	GridTie.Idle	telemetry-healthy + grid-tie reported	enable Acquire Telemetry; arm 1 Hz tick
Initializing	Fault.TelemetryFault	telemetry timeout at startup	emit <code>event_alert</code> (startup-fault)
GridTie.Idle	GridTie.Diverting	<code>net_grid_power < -threshold</code>	start raising battery <code>max_charge_current</code>
GridTie.Diverting	GridTie.Idle	<code> net_grid_power < deadband</code>	hold current setpoint
GridTie.Diverting	GridTie.Holding	<code>battery_soc >= max_soc</code>	clamp <code>max_charge_current</code> to zero
GridTie.Holding	GridTie.Diverting	surplus persists + <code>battery_soc < max_soc - hysteresis</code>	resume diversion
GridTie.Idle	GridTie.Discharging	net-import in night-rate window	set Victron <code>grid_setpoint</code> to track ≈ 0
GridTie.Discharging	GridTie.Idle	<code>battery_soc ≤ min_soc</code> OR sun rising	clear <code>grid_setpoint</code> ; resume normal Idle
GridTie.*	Island.BatteryDischarge	Victron reports <code>mode == island</code>	disable Cloud Forward; clamp Dispatch Commands to safe defaults
Island.BatteryDischarge	Island.SolarAssist	Sun rising + Victron AC-couples inverters	observe (no commands; Victron's frequency-shift drives)
Island.SolarAssist	Island.BatteryDischarge	Sun setting OR battery saturating	(state change only; Victron handles physics)
Island.*	GridTie.Idle	Victron reports <code>mode == grid-tie</code>	re-enable Cloud Forward (if configured)
GridTie.* / Island.*	Fault.<category>	fault event of category	emit <code>event_alert</code> (fault); clamp to safe defaults

From	To	Event (trigger)	Action (side effect)
Fault.*	GridTie.* OR Island.*	fault cleared + Victron mode	resume normal operation in matching mode

Process controls

Per Decision 7 in `proposal.md`. The CSPEC activates / deactivates sibling level-1 processes based on current mode:

Mode	Dispatch Commands	Cloud Forward
Initializing	deactivated	deactivated
GridTie (any sub-state)	activated	activated <i>(if owner-enabled)</i>
Island	activated <i>(limited safe setpoints)</i>	deactivated <i>(no remote forward during outage)</i>
Fault	activated <i>(safe defaults; clamp commands)</i>	deactivated

Tick

Per Decision 4 — **hybrid event-driven + 1 Hz tick**. The tick lets the CSPEC re-evaluate policy on schedule (catch slow drift, expire stale events, periodically re-emit alerts if conditions persist).

Cross-links

- ↑ Parent diagram: [Level-1 DFD](#) (Energy Manager is bubble #2)
- ↑ Lock context: [proposal.md](#) · [naming-review.md](#)
- 📖 Dictionary: [../../../../dictionary.yaml](#) (state entries under `level: 2`)
- 📖 HP reference: [Control Specification](#) · [State Transition Diagram](#) · [State Charts \(Harel\)](#)

Next sub-stage

Event glossary — formalize every transition trigger above into a `event_*` stable ID in the dictionary with type, source, and semantics. Then **action specifications** for the side-effects. After both: the Energy Manager CSPEC is complete and we move to PSPECs for the level-1 leaf bubbles (Stage 4).

Stage 4 — PSPECs

5 PSPEC(s)

PSPEC — Acquire Telemetry

Process: [proc_acquire_telemetry](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_f1_inverter_telemetry</code> — F1: per-channel telemetry	<code>term_inverters</code>	Sub-1G RF → DTU-Pro-S or OpenDTU
<code>flow_f3_net_grid_power</code> — F3: net grid power, V/I/PF	<code>term_meter</code>	RS485 Modbus RTU
<code>flow_f4_battery_state</code> — F4: SoC, mode, AC-in	<code>term_battery_system</code>	MQTT / Modbus TCP
<code>flow_event_config</code> — <code>event_config</code>	<code>proc_handle_user_input</code>	—

OUTPUTS

Flow	To
<code>flow_telemetry_to_state</code> — normalized state	<code>store_system_state</code>

TRANSFORMATION (textual)

```
Each ingest cycle:
  Read F1 PER CHANNEL TELEMETRY from the inverter side via the
  configured DTU adapter (DTU-Pro-S, OpenDTU, or AhoyDTU).
  Read F3 NET GRID POWER V I PF from the grid-tied power meter
  via Modbus RTU.
  Read F4 SOC MODE AC IN from the battery system via MQTT or
  Modbus TCP.
On receipt of EVENT CONFIG:
  Apply the new configuration values (sample rates, source
  selection, scaling).
Normalize all readings into the canonical NORMALIZED STATE shape
and write it to store_system_state.
```

COMPUTATIONAL CONSTRAINTS

- **Frequency:** ≥ 1 Hz aggregate ingest rate; ≥ 0.5 Hz per source
- **Timing:** End-to-end ingest → `store_system_state` write latency < 1 s

VERIFICATION

- **Methods:** test, analysis

- **Coverage target:** 90.0%
- **Validation scenarios:**
 - Sustained 1 Hz aggregate ingest across all three sources for 24 hours
 - DTU reconnect within 30 s of a transient network drop
 - Modbus RTU CRC failure causes only that sample to be discarded, not session drop

OBSERVABILITY

Metrics:

- `telemetry_samples_total` *counter* — Total telemetry samples ingested (all sources)
- `telemetry_ingest_seconds` *histogram* (s) — End-to-end ingest cycle duration
- `telemetry_source_lag_seconds` *gauge* (s) — Per-source age of latest sample

Traces:

- `telemetry.ingest_cycle` — One full multi-protocol ingest cycle

Log categories:

- `telemetry.adapter` (*level: info*)

Alerts:

- `telemetry_source_stale` (*warning*) — when `max(telemetry_source_lag_seconds) > 30`
→ [runbook](#)

COMMENTS

Sole writer into the telemetry portion of the store. Vendor adapter details and reconnection policy live in the architecture model, not here (PSPECs specify what, not how — 1988 §13.1).

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../../../../toolkit/PSPEC_DESIGN.md](#) .

PSPEC — Dispatch Commands

Process: [proc_dispatch_commands](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_cmd_setpoint</code> — <code>cmd_setpoint</code>	<code>proc_compute_balance</code>	—
<code>flow_cmd_inverter_limit</code> — <code>cmd_inverter_limit</code>	<code>proc_compute_balance</code>	—

OUTPUTS

Flow	To
<code>flow_f2_power_limit</code> — F2: power-limit setpoints	<code>term_inverters</code>
<code>flow_f5_battery_command</code> — F5: max-charge, grid setpoint	<code>term_battery_system</code>

TRANSFORMATION (textual)

```
On receipt of CMD SETPOINT:
  Translate the desired inverter power-limit setpoint into the
  vendor-specific frame and issue F2 POWER LIMIT SETPOINTS to
  the inverter side via the configured DTU adapter.
On receipt of CMD INVERTER LIMIT:
  Translate the desired battery charge / grid-setpoint command
  and issue F5 MAX CHARGE GRID SETPOINT to the battery system
  via MQTT publish.
```

COMPUTATIONAL CONSTRAINTS

- **Accuracy:** Translated values honor command to within vendor protocol resolution
- **Timing:** Command → actuator latency < 500 ms

COMMENTS

Adapter logic. The "issue" keyword in the body marks F2 and F5 as time-transient (1988 §13.3) — each command propagates once, not held by this bubble.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../../../../toolkit/PSPEC_DESIGN.md](#).

PSPEC — Serve UI

Process: [proc_serve_ui](#) (level-1 DFD) Generated from `dictionary.yaml` . Do not hand-edit.

INPUTS

Flow	From	Medium
<code>flow_state_to_ui</code> — <code>system_state</code>	<code>store_system_state</code>	—
<code>flow_event_alert</code> — <code>event_alert</code>	<code>proc_compute_balance</code>	—

OUTPUTS

Flow	To
<code>flow_f7_user_view</code> — F7: dashboards, alerts	<code>term_user</code>

TRANSFORMATION (textual)

Render F7 DASHBOARDS ALERTS combining:
 the current SYSTEM STATE (power flows, SoC, mode, recent history),
 and any EVENT ALERT received from the Energy Manager.
 Provide drill-in views for inverter-side, grid-side, and battery-side detail.

COMPUTATIONAL CONSTRAINTS

- **Frequency:** Dashboard refresh ≥ 1 Hz; alert latency < 500 ms

COMMENTS

Read-only presentation. No commands originate here; user actions go through `proc_handle_user_input`.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../.../toolkit/PSPEC_DESIGN.md](#) .

PSPEC — Handle Input

Process: [proc_handle_user_input](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_f6_user_input</code> — F6: config, override	<code>term_user</code>	UI

OUTPUTS

Flow	To
<code>flow_event_override</code> — event_override	<code>proc_compute_balance</code>
<code>flow_event_config</code> — event_config	<code>proc_acquire_telemetry</code>

TRANSFORMATION (textual)

```
On receipt of F6 CONFIG OVERRIDE from the angler / owner:
  Classify the input:
    - structural changes (setpoints, source enable/disable,
      policy thresholds) → issue EVENT CONFIG;
    - immediate operational overrides (manual mode, enable/
      disable, force-recompute) → issue EVENT OVERRIDE.
  Apply input validation; reject malformed inputs.
```

COMPUTATIONAL CONSTRAINTS

- **Timing:** Input → emitted event latency < 200 ms

COMMENTS

Owner-facing input gateway. Both EVENT CONFIG and EVENT OVERRIDE are transient (1988 §13.3) — the "issue" keyword reflects that.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../.../toolkit/PSPEC_DESIGN.md](#).

PSPEC — Cloud Forward

Process: [proc_cloud_forward](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_state_to_cloud</code> — <code>system_state</code>	<code>store_system_state</code>	—

OUTPUTS

Flow	To
<code>flow_f8_cloud_forward</code> — F8: optional telemetry forward	<code>term_smiles_cloud</code>

TRANSFORMATION (textual)

```
If cloud forwarding is enabled by configuration:
  Subscribe to changes in SYSTEM STATE and issue
  F8 OPTIONAL TELEMETRY FORWARD to S-Miles Cloud at the
  configured cadence.
Otherwise, do nothing.
```

COMPUTATIONAL CONSTRAINTS

- **Frequency:** Configurable; default 1/min when enabled
- **Timing:** Best-effort; not in the diversion-control critical path

COMMENTS

Optional bubble (per Stage 1 Decision 5). The "issue" keyword marks F8 as time-transient — each posting is a single event, not a held output (1988 §13.3).

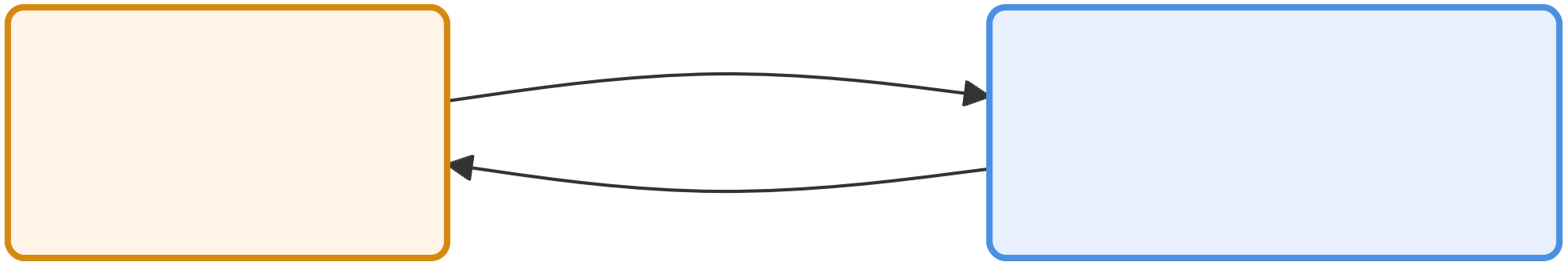
Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../.../toolkit/PSPEC_DESIGN.md](#).

Stage 5 — Architecture

2 module(s), 1 interconnect(s)

AFD (Flow Diagram)



AID (Interconnect Diagram)



Modules

AMS — Controller Host (AM 1)

Module: [am_controller_host](#) *Generated from `dictionary.yaml`. Do not hand-edit.*

DESCRIPTION

Single-board computer running Linux + the HP-derived controller service. Carries all the device-facing protocols (Sub-1G RF to the DTU, Modbus RTU to the meter, MQTT/Modbus TCP to the battery system), the diversion-control state machine, and the output adapters that translate `cmd_setpoint` / `cmd_inverter_limit` back into vendor protocols.

CROSS-REFERENCE (allocation)

Requirements component	Kind
<code>proc_acquire_telemetry</code>	process
<code>proc_dispatch_commands</code>	process
<code>proc_cloud_forward</code>	process
<code>proc_compute_balance</code>	process (state-rich; CSPEC lives here)
<code>store_system_state</code>	data store

DESIGN RATIONALE

Single-host architecture chosen over a distributed setup because the dataset is small enough (sub-kHz aggregate) to fit on one modest controller, and a single failure domain simplifies the diversion-control safety story.

DESIGN JUSTIFICATION

Pi 4 with 2 GB RAM is overprovisioned for the workload: ~50 KB/s aggregate ingest, ~100 KB heap working set. Vendor adapters are Python or Go processes well within budget.

REQUIRED CONSTRAINTS

- **Reliability:** MTBF > 8000 hours indoor mounted.
- **Safety:** Diversion command path must fail-safe to 'zero export' on host failure (Victron's fallback handles outage transfer independently).
- **Physical:** Indoor mount; passive cooling; mains UPS.
- **Cost:** BoM ≤ \$200 including UPS and enclosure.

INTERFACES

Inputs: F1 PER CHANNEL TELEMETRY (Sub-1G RF via DTU-Pro-S or OpenDTU); F3 NET GRID POWER (RS485 Modbus RTU); F4 SOC MODE AC IN (MQTT or Modbus TCP). Outputs: F2 POWER LIMIT SETPOINTS (OpenDTU only); F5 MAX CHARGE GRID SETPOINT (MQTT

publish); F8 OPTIONAL TELEMETRY FORWARD (HTTPS, off by default). LAN: WebSocket + HTTP to the Dashboard Web App.

BUDGETS (allocations to this module)

Budget	Unit	This module	System target	Reserve
<code>budget_diversion_loop_latency</code> — Grid sense → inverter setpoint latency	ms	800.0	1000.0	200.0
<code>budget_monthly_cloud_cost</code> — Monthly cloud cost	USD	3.0	5.0	1.0

TPMs (tracking this module's budgets)

TPM	Unit	Current	Growth allowance	Threshold
<code>tpm_diversion_response_p99</code> — Diversion loop p99 response latency (measured)	ms	750.0	250.0	1000.0
<code>tpm_actual_monthly_cost</code> — Actual monthly cloud cost (last 30d)	USD	2.3	2.7	5.0

SLOs (apply to this module)

SLO	Target	Window	Error budget
<code>slo_diversion_loop_latency</code> — Diversion-loop response latency	1.0 seconds	30d	0.5%
<code>slo_monthly_cost</code> — Monthly cloud cost SLO	5.0 USD	30d	20.0%

Format: 2000 §4.2.5.4 — typical AMS contents. See [../ARCH_DESIGN.md](#).

AMS — Dashboard Web App (AM 2)

Module: [am_dashboard_app](#) *Generated from `dictionary.yaml`. Do not hand-edit.*

DESCRIPTION

SPA served from the controller host. Renders live dashboards (power flows, SoC, mode, recent history); surfaces alerts in a banner; exposes config controls and manual override buttons. Runs entirely in the owner's browser; talks back via WebSocket and HTTP.

CROSS-REFERENCE (allocation)

Requirements component	Kind
<code>proc_serve_ui</code>	process
<code>proc_handle_user_input</code>	process

DESIGN RATIONALE

Browser-based UI chosen over a native app to avoid app-store friction and to make the controller usable from any device on the owner's LAN without install.

REQUIRED CONSTRAINTS

- **Reliability:** Reconnects to controller within 5 s of WebSocket drop; surfaces 'disconnected' state to the owner.
- **Physical:** Runs in any modern browser (Chrome, Safari, Firefox).

BUDGETS (allocations to this module)

Budget	Unit	This module	System target	Reserve
<code>budget_monthly_cloud_cost</code> — Monthly cloud cost	USD	1.0	5.0	1.0

TPMs (tracking this module's budgets)

TPM	Unit	Current	Growth allowance	Threshold
<code>tpm_actual_monthly_cost</code> — Actual monthly cloud cost (last 30d)	USD	2.3	2.7	5.0

SLOs (apply to this module)

SLO	Target	Window	Error budget
slo_monthly_cost — Monthly cloud cost SLO	5.0 USD	30d	20.0%

Format: 2000 §4.2.5.4 — typical AMS contents. See [../../ARCH_DESIGN.md](#).

Interconnects

AIS — Local LAN

Interconnect: [ai_local_lan](#) **Endpoints:** `am_controller_host`, `am_dashboard_app`

Generated from `dictionary.yaml`. Do not hand-edit.

DESCRIPTION

The owner's local network connecting the dashboard browser to the controller host. WebSocket carries push notifications (state updates, alerts); HTTP serves the SPA and accepts config POSTs.

CARRIES

Architecture flows allocated to this channel:

- `af_state_to_dashboard` — state + alerts (data)
- `af_input_to_controller` — config + override (data)

PROTOCOL STANDARD

HTTP/1.1 (RFC 7230) for static asset serving and config POSTs; WebSocket (RFC 6455) for push notifications. JSON message format documented in the controller's protocol spec.

DESIGN RATIONALE

LAN-only by default — no public Internet exposure. Owner can reverse-proxy externally if remote access is wanted, but that is out-of-scope here.

REQUIRED CONSTRAINTS

- **Reliability:** Survives router reboots without manual reconnect on the dashboard side.

THREAT MODEL (STRIDE)

Per Microsoft SDL (Howard & Lipner 2006). Each row is the narrative describing how the threat category is addressed.

Category	Mitigation
Sp spoofing	Shared-secret authentication (rotating per-owner bearer token, served by the controller's first-login bootstrap) prevents unauthorized browsers from connecting. The LAN itself is the outer ring of defense — assumes the owner has reasonable Wi-Fi hygiene.
Tampering	TLS (HTTPS + WSS) protects in-flight messages against tampering. Controller signs config-change responses with the session token so the browser can detect mid-session injection.
Repudiation	<code>out_of_scope</code> — single owner; no audit requirement at this tier. ADR-002 explicitly defers any audit trail to a future cloud forwarding option.

Category	Mitigation
Info disclosure	TLS encryption protects state + telemetry in transit. Dashboard does not store usage data persistently in the browser beyond the session token.
Denial of service	Rate-limit config-POST endpoints (10 req/min per session). WebSocket connection cap of 4 per session. Controller's diversion-control loop is decoupled from dashboard liveness — loss of LAN does not affect energy management.
Elev. of privilege	Dashboard session token grants only configured-scope permissions (config + view; no firmware update, no factory reset). Reset and firmware update require a separate physical-button flow on the controller host.

Format: 2000 §4.2.6.2 — typical AIS contents. See [../../../../ARCH_DESIGN.md](#).

Modernization

ADRs

ADR — Outage handling lives inside the Energy Manager CSPEC

ID: `adr_001_outage_inside_cspeg` **Status:** Accepted **Date:** 2026-05-22 **Author:** solar dogfood

Generated from `dictionary.yaml` . Do not hand-edit.

CONTEXT

The system needs to handle grid outages (utility goes down → battery takes over → utility returns) without dropping into a fault state. Two approaches were on the table during Stage 2: (a) make outage handling a separate process bubble that gates the diversion loop; (b) make outage handling a mode of the Energy Manager's CSPEC alongside GridTie and Fault.

DECISION

Outage handling is an `Island` mode of the Energy Manager CSPEC, peer to GridTie and Fault. The diversion-loop sub-states live under GridTie; the Island mode has its own setpoint-passthrough behavior; transitions are driven by Victron's reported mode.

CONSEQUENCES

- The CSPEC is the single source of truth for "what mode is the system in" — no separate outage-state bubble to keep in sync.
- Victron is trusted as the authoritative mode source; we don't debounce its mode reports (the AC transfer is < 20 ms, fast enough that our 1 Hz tick has nothing meaningful to add).
- Future "manual override" behavior is a modifier *within* a mode (Decision 5), not a separate Manual mode.

ALTERNATIVES CONSIDERED

- Separate outage-handler process bubble — rejected: forces every level-1 reader to gate on two states.
- Manual mode as a peer to GridTie/Island/Fault — rejected: doubles state count and pairs poorly with diversion sub-states.

AFFECTS

- **Modules:** `am_controller_host`
- **Processes:** `proc_compute_balance`
- **States:** `state_grid_tie`, `state_island`, `state_fault`

Format: Michael Nygard 2011 — Context / Decision / Consequences / Alternatives. See [../toolkit/MODERNIZATION_DESIGN.md](https://toolkit/MODERNIZATION_DESIGN.md) .

ADR — Dashboard is local-LAN only by default

ID: `adr_002_local_only_dashboard` **Status:** Accepted **Date:** 2026-05-22 **Author:** solar dogfood

Generated from `dictionary.yaml` . Do not hand-edit.

CONTEXT

The owner needs a dashboard to view solar production, battery state, and recent events; and to push config + manual overrides. Public remote access is a tempting feature but adds significant attack surface (any control-plane mistake becomes internet-exposed) and compliance considerations (CCPA disclosure for usage telemetry).

DECISION

Dashboard runs as a browser-side SPA served by the controller host over the owner's local LAN. No public-internet exposure by default. Owner-led reverse-proxy setup is documented for remote access as an opt-in, but it is out of scope for the supported configuration.

CONSEQUENCES

- Reduced attack surface: dashboard authentication can use a shared secret on LAN rather than full OAuth/OIDC.
- Owners who want remote access must run their own reverse proxy and assume responsibility for its security posture.
- Cloud Forward (the optional S-Miles bridge) is unaffected — it's a separate optional outbound path with its own threat model.

ALTERNATIVES CONSIDERED

- Public cloud-hosted dashboard with per-user OAuth — rejected for now (compliance + scope).
- VPN-only access — rejected: too high friction for typical owners.

AFFECTS

- **Modules:** `am_dashboard_app` , `am_controller_host`
- **Interconnects:** `ai_local_lan`

CATALOG REFERENCES

MITRE ATT&CK: [T1190](#) , [T1078](#)

CWE: [CWE-306](#) , [CWE-319](#)

Compliance: `CCPA-1798.100`

Format: Michael Nygard 2011 — Context / Decision / Consequences / Alternatives. See [../toolkit/MODERNIZATION_DESIGN.md](#).

Solar Local Stack — Service Level Objectives

Generated from `dictionary.yaml`. Do not hand-edit.

SLOs commit external promises about the runtime behavior of the architecture model. Each declares the SLI being measured, the target value, the rolling window, and the error budget that gates release decisions (Google SRE Book 2016).

slo_diversion_loop_latency

Diversion-loop response latency

- **Target:** 1.0 seconds
- **Window:** 30d
- **Error budget:** 0.5%
- **SLA (customer-facing):** Diversion-loop p99 response under 1 second over any 30-day window.

SLI

p99 latency from grid-sense reading to inverter setpoint command.

```
histogram_quantile(0.99, rate(diversion_loop_seconds_bucket[5m]))
```

Applies to

- **Modules:** `am_controller_host`

Derived from TPM

[tpm_diversion_response_p99](#) — Diversion loop p99 response latency (measured) (current 750.0 ms)

Runbook on budget burn

[runbooks/slo-diversion-burn.md](#)

slo_monthly_cost

Monthly cloud cost SLO

- **Target:** 5.0 USD
- **Window:** 30d
- **Error budget:** 20.0%

SLI

Sum of all cloud-cost-explorer line items tagged solar-stack.

```
sum(cloud_cost_usd_monthly_total)
```

Applies to

- **Modules:** `am_controller_host`, `am_dashboard_app`

Derived from TPM

`tpm_actual_monthly_cost` — Actual monthly cloud cost (last 30d) (current 2.3 USD)

See [../toolkit/MODERNIZATION_DESIGN.md](#) §4.3 — *SLI/SLO/SLA chain*.

Bounded Contexts

Context Map (Mermaid SVG)



Runbooks

Runbook — diversion-loop SLO error budget burning

SLO: `slo_diversion_loop_latency` **Trigger:** error budget burn rate above 2× normal over the trailing 6 hours.

SYMPTOMS

- p99 diversion-loop response latency rising above 1 s target.
- Risk of grid-export events during surplus-solar bursts (cloud-clearing, sunrise spikes).

DIAGNOSIS

1. **Check telemetry ingest stage.** If `telemetry_ingest_seconds` p99 is climbing, the bottleneck is on the read side — see `telemetry-source-stale.md`.
2. **Check Energy Manager CSPEC cycle time.** If state-machine evaluation is slow, may be GC pause (controller is Python) or excessive logging in the diversion-control loop.
3. **Check dispatch path.** If `cmd_setpoint` → DTU write is taking long, check DTU connection quality.
4. **Check host load.** Pi 4 thermal throttling (~80°C) will slow the loop; ensure passive cooling is adequate.

RESOLUTION

- **Ingest bottleneck:** tune source polling intervals; reduce simultaneity if necessary.
- **CSPEC cycle:** review recent code changes to `proc_compute_balance`; profile and optimize the diversion-loop tick path.
- **DTU dispatch slow:** if OpenDTU, check WiFi quality; if DTU-Pro-S, verify Sub-1G channel isn't congested.
- **Thermal:** add or improve passive cooling; document if owner's mounting location is thermal-marginal.

ESCALATION

If error budget continues burning at >2× over 24 hours, notify the owner: there's a real risk of grid-export events under sunlight surges. Consider engaging Victron's onboard fallback (zero-export mode) as a stopgap.

Runbook — telemetry_source_stale

Alert: `telemetry_source_stale` **Severity:** warning **Module:** `am_controller_host`
(`proc_acquire_telemetry`) **Triggered when:** `max(telemetry_source_lag_seconds) > 30`

SYMPTOMS

- One or more telemetry sources (inverter, meter, battery) has gone silent.
- `store_system_state` becomes stale for the affected source's portion.
- Energy Manager CSPEC may degrade to conservative behavior (no diversion, no discharge).

DIAGNOSIS

Identify which source by checking `telemetry_source_lag_seconds{source="..."}`:

1. **Inverter side stale (DTU):** check DTU power + Sub-1G antenna; ping the DTU's web UI.
2. **Meter side stale (Modbus RTU):** check RS485 wiring + termination; ping Modbus address; check RTU baud rate config.
3. **Battery side stale (MQTT / Modbus TCP):** check Cerbo GX is online and MQTT broker reachable.

RESOLUTION

- **DTU reachable but no telemetry:** restart DTU service (Hoymiles-specific procedure).
- **DTU offline:** power-cycle the DTU; if persistent, check antenna position.
- **Meter offline:** verify RS485 termination; check meter is powered.
- **Battery offline:** check Cerbo GX network; verify MQTT broker logs.

ESCALATION

After 15 minutes without resolution, page `#oncall-solar-team`. If the diversion loop is degraded (cannot read all three sources), notify the owner that zero-export compliance may not be enforced — they may want to disable the inverter manually until restored.

Appendix — HP Quick Reference

Hatley-Pirbhai Quick Reference

Concise refresh on HP vocabulary. Each entry: one-line definition · modern analog where helpful · tiny example · cross-links to related terms.

Anchor links work from chat: e.g. `[DFD](toolkit/reference/HP_QUICK_REF.md#dfd--data-flow-diagram)`. Headings use kebab-case anchors automatically.

Sourced from the 1988 book (*Strategies for Real-Time System Specification*, Hatley & Pirbhai) and the 2000 book (*Process for System Architecture and Requirements Engineering*, Hatley, Hruschka & Pirbhai). Some entries note where they were introduced.

1. The Two (then Three) Models

Requirements Model

The **"what"** half of HP — what the system must do, independent of how. Composed of: a Process Model (DFDs + PSPECs), a Control Model (CFDs + CSPECs), a Requirements Dictionary, and Timing Specs. Modern analog: a behavior spec + state machine + interface contracts, all kept consistent. *See also:* [Architecture Model](#), [Essential vs Implementation Model](#).

Architecture Model

The **"how"** half — physical/logical realization of the requirements. Composed of: Architecture Flow Diagrams (AFDs), Architecture Interconnect Diagrams (AIDs), Architecture Module Specs (AMS), Architecture Interconnect Specs (AIS), Architecture Dictionary. Modern analog: deployment diagram + component diagram + interface contracts + the wiring spec. *See also:* [Architecture Templates](#).

Mechanisms Model (added in 2000)

A **middle layer between Requirements and Architecture**, introduced in the 2000 book. Captures *generic solution patterns* (control loops, state-machine engines, fault-handling schemes) before committing to specific hardware/software modules. Modern analog: design patterns — but named, traceable, and explicit. *See also:* [Layered Models](#), [Enhanced Requirements Model](#).

Essential vs Implementation Model

HP's foundational distinction. **Essential** = what the system must do regardless of technology (the "perfect" technology-independent view). **Implementation** = how it's actually built with real components. The Architecture Model is the bridge. Modern analog: domain model vs deployment model.

Layered Models

The 2000 book's three-tier framing: Essential Requirements Model → Mechanisms Model → Architecture Model. Each layer refines the previous without losing intent. *See also:* [Mechanisms Model](#).

Enhanced Requirements Model

The 2000 book's name for the requirements model *after* the four architecture templates have been applied. Refines the essential model with the bookkeeping the implementation will need (I/O, UI, self-test). *See also:* [The Four Architecture Templates](#).

2. Notation Primitives (what you draw)

Process (Bubble)

The basic data-transformation unit. Drawn as a **circle** with a name and a hierarchical number (e.g., "2.3 Validate Coin"). Each non-leaf bubble decomposes into a child DFD; each leaf bubble has a PSPEC. Modern analog: a function (leaf) or a module (non-leaf).

Terminator

An **external entity** that sits *outside* the system boundary but exchanges data or control with it. Drawn as a **rectangle**. People, other systems, sensors, actuators, regulators. Modern analog: external actor on a context diagram, or "external service" in a service map. *See also:* [Context Diagram](#).

Data Store

A **place data is held** between processes (file, database, table, queue, buffer). Drawn as **two parallel lines** with a name between. Modern analog: a database table, a Redis key, a Kafka topic — anything where one process writes and another later reads.

Control Store

Same shape as a data store, but for **control information** (state values, mode flags). Less common in practice; usually merged with state machines inside CSPECs. *See also:* [CSPEC](#).

Data Flow

A **labeled arrow** from a source (process, store, terminator) to a destination, carrying named data. Solid line. Modern analog: a function argument, a message on a queue, a struct passed somewhere.

Control Flow

Like data flow but carries **events, triggers, or state signals** that *cause* processes to do something. Drawn with a **dashed line** in classic HP. Modern analog: an event, an interrupt, a signal, a webhook.

CSPEC Bar (Short Unlabeled Bar)

A small **horizontal bar** drawn into a DFD that represents a CSPEC reaching into the data world to **activate, deactivate, trigger, or enable** processes. The "magic" notation that connects control to data without cluttering the DFD. Modern analog: the rules engine reaching in to enable/disable handlers based on state. *See also:* [Process Controls](#).

Process Controls

The actual signals the CSPEC bar emits into processes — **activator** (start running), **deactivator** (stop), **trigger** (run once). Modern analog: lifecycle hooks (`start()` , `stop()` , `tick()`).

3. Diagrams

Context Diagram (DCD / CCD / ACD)

The **level-0 diagram** — single bubble (the system) plus all terminators around it, with flows crossing the system boundary. Three variants: **DCD** = Data Context Diagram (data flows only), **CCD** = Control Context Diagram (control flows only), **ACD** = Architecture Context Diagram (the architecture-side equivalent). *Drawn first; the methodology refuses to go deeper until it's locked.* Modern analog: a system context diagram in C4 architecture notation.

DFD — Data Flow Diagram

The **bread and butter** of HP. Bubbles (processes) connected by data flows, with terminators around the edge and data stores between processes. Hierarchical — each non-leaf bubble has a child DFD that decomposes it. Modern analog: a structured flowchart-of-transformations, or a Petri-net-like view of data movement. *See also:* [Leveling](#), [Balancing](#).

CFD — Control Flow Diagram

The **partner to a DFD** for the control-flow dimension. Shows control flows between processes and to/from CSPEC bars. Often drawn on the same sheet as the corresponding DFD. Modern analog: the signaling layer of an event-driven system.

AFD — Architecture Flow Diagram

The **architecture-side counterpart to a DFD**. Shows physical/logical *modules* (rounded rectangles) connected by *architecture flows* (which group the underlying data/control flows). Modern analog: a deployment diagram showing what runs where. *See also:* [AID](#), [Architecture Module](#).

AID — Architecture Interconnect Diagram

Shows the **physical channels** (buses, network links, RS-485, MQTT, file paths) carrying architecture flows between modules. **AID + AFD together = the full architecture picture.** Modern analog: the wiring/network diagram for a deployment. *See also:* [AIS](#).

AICD / AFCD (added in 2000)

Architecture-side **context diagrams** — same idea as DCD/CCD but for the architecture model. AICD = Architecture Interconnect Context Diagram (level-0 of AID). AFCD = Architecture Flow Context Diagram (level-0 of AFD).

4. Specifications (the textual artifacts)

PSPEC — Process Specification

The **leaf-level functional contract** for a bubble that doesn't decompose further. Specifies: inputs consumed, outputs produced, preconditions, postconditions, the transformation rule. Often written in structured English or a decision table. Modern analog: a docstring-plus-contract; a JSDoc/PEP-257 with formal pre/post conditions. *Required for every leaf bubble. See also: [Process \(Bubble\)](#), [Primitive \(Leaf Bubble\)](#).*

CSPEC — Control Specification

The **state/sequencing logic** that controls when processes activate. Usually contains a state-transition diagram, a decision table, or both. Reaches into the DFD via the CSPEC bar to activate/deactivate/trigger processes. Modern analog: a state machine spec, a rules engine config, an FSM library setup. *See also: [STD](#), [Decision Table](#), [CSPEC Bar](#).*

TSPEC — Timing Specification

A **formal capture of timing requirements**: response times, repetition rates, deadlines, latencies. First-class artifact, not just a comment. Example: "F3 (net grid power read) latency ≤ 1 s; sampling rate ≥ 1 Hz." Modern analog: SLO definitions, but at the requirements level. *See also: [Timing Requirements](#).*

AMS — Architecture Module Specification

Like a PSPEC but for an **architecture module** (a deployable component). Describes the module's responsibilities, the data/control flows it handles, its internal allocations. Modern analog: a service README / component contract.

AIS — Architecture Interconnect Specification

Like a PSPEC but for an **interconnect** — a physical/logical channel. Specifies the protocol, format, timing, framing, error handling. **This is HP's answer to "interfaces lost in implementation"** — interfaces are first-class artifacts. Modern analog: a protocol spec, an OpenAPI document, a gRPC IDL, a Modbus register map.

Requirements Dictionary

The **central glossary of every named flow, store, and data element** in the requirements model. Every arrow/store/element on every diagram has an entry. Modern analog: a data dictionary, schema registry, or shared types library. *See also: [Group Structure](#), [Primitive Attributes](#).*

Architecture Dictionary

The dictionary's architecture-side counterpart — every architecture module, flow, and interconnect has an entry.

Group Structure

A way of specifying **composite data elements** in the Requirements Dictionary (e.g., "telemetry_packet = timestamp + channel_id + voltage + current"). Modern analog: a struct/record type definition.

Primitive Attributes

The leaf-level attributes in the dictionary — units, ranges, precision, encoding. Modern analog: a primitive type definition with constraints.

5. Concepts & Discipline

Leveling

The **hierarchical decomposition** of DFDs. Level-0 = context diagram (one bubble). Level-1 = the first decomposition (3-7 bubbles). Each non-leaf bubble at level N becomes its own DFD at level N+1. The leveling rule: **every bubble at the same level is at the same level of abstraction**. *Anti-rathole mechanism: you can't go deep on one bubble while leaving siblings under-specified.*

Balancing

The conservation law of DFDs. **Flows in and out of a parent bubble must match flows in and out of its child DFD**. A child DFD must preserve the parent's interface exactly. *Modern analog:* a function's signature must be consistent with what its body actually consumes and produces. Validation: balancing checks find drift early.

Decomposition

The process of taking one bubble and breaking it into a child DFD with multiple bubbles. HP demands that decomposition be *justified* — typically because the bubble has 3+ distinct responsibilities, or its PSPEC would be too long. *See also:* [Leveling](#).

Seven-Plus-or-Minus-Two

HP's rule of thumb: **a DFD should have 5-9 bubbles**. Below 5, you haven't decomposed enough. Above 9, the diagram is incomprehensible. Sourced from Miller's classic cognitive-load paper. Modern analog: the "team size of two pizzas" or "module file under 500 lines" heuristic.

Traceability

The **explicit links** from requirements to architecture to implementation (and ideally to tests). HP demands that every requirement be allocated to an architecture module, and every architecture module justify itself against requirements. Modern analog: requirement-IDs on commits, story-to-test mapping, but enforced as a first-class artifact. *See also:* [Traceability Matrix](#), [Extended Traceability](#).

Traceability Matrix

The **table** that records traceability links — typically requirements as rows, architecture modules as columns, with cells marking allocations. *Anti-85%-syndrome mechanism: uncovered requirements stand out as empty rows.*

Primitive (Leaf Bubble)

A bubble that **doesn't decompose further** — its behavior is captured in a PSPEC instead of a child DFD. The bottom of the hierarchy. Modern analog: a pure function with a docstring.

System Hierarchy / Universal Hierarchy

HP's view that all systems exist in a containing hierarchy — your system is part of a larger system (the *environment*), and contains smaller systems (the *subsystems*). The context diagram defines the boundary between *your* system and the level above it. *See also:* [Multiple Hierarchies](#).

Transaction Center

A bubble that **dispatches between multiple downstream processes based on input** (often command/event-driven). Modern analog: a switch statement, a router, a dispatcher.

Primitive Network

The DFD at its **finest level of decomposition** — only leaf bubbles remain, all with PSPECs, no further decomposition possible. Modern analog: the "leaf functions" of a refactored codebase.

6. The Four Architecture Templates

The 1988 book's contribution: every architecture wraps the requirements model in **four standard "process blocks" surrounding a core**. These are the things real systems need that pure structured analysis ignored.

Input Processing

The **block that handles incoming data/control from terminators** — sensor reads, network input, button presses, file reads. Decouples the rest of the system from physical-input concerns. Modern analog: input adapters in hexagonal architecture.

Output Processing

The **block that handles outgoing data/control to terminators** — actuator commands, displays, network output, file writes. Modern analog: output adapters in hexagonal architecture.

User Interface Processing

The **block that handles human interaction** — dashboards, configuration, manual override, alerts. Separate from generic I/O because human latency, error tolerance, and presentation needs differ. Modern analog: presentation layer.

Maintenance / Self-Test / Redundancy Management

The **block that handles non-functional concerns** — health checks, diagnostics, failover, recovery, redundancy. The "what keeps this thing running" layer. Modern analog: SRE/observability + HA features.

Architecture Template (4 surrounding blocks)

The collective name. Every architecture is the **requirements model "core" surrounded by these four blocks**. Forces you to think about I/O, UI, and self-test as first-class architecture concerns, not afterthoughts.

7. Control Formalisms (inside CSPECs)

FSM — Finite State Machine

Classic state machine: states + transitions + events + actions. HP uses FSMs inside CSPECs for any process with non-trivial sequencing. *See also:* [STD](#), [State Transition Matrix](#).

STD — State Transition Diagram

The **graphical form of a state machine** — circles for states, arrows for transitions labeled with event/action pairs. Modern analog: a state-chart in XState or similar.

State Transition Matrix / Table

The **tabular form of a state machine** — rows are current states, columns are events, cells are next-state-and-action. Equivalent to STD, sometimes more compact for large state spaces.

Decision Table

A **tabular form for combinational logic** (no state). Rows are conditions, columns are rule combinations, cells say which actions fire. Modern analog: a rules engine ruleset, a truth table.

Combinational vs Sequential Machine

Combinational = output depends only on current inputs (decision table). **Sequential** = output depends on inputs *and* current state (STD/state matrix). HP CSPECs may contain either or both.

State Chart (Harel, added in 2000)

A **hierarchical state machine** with nested states and parallel regions. From David Harel's 1987 work. The 2000 book brings these into HP for cases where a flat FSM is too noisy.

Composite CSPEC / Multi-Sheet CSPEC

A **CSPEC that spans multiple sheets** because the control logic is too complex for one page. Same logic, just paginated.

8. 2000-Book Additions

The 2000 book (Hatley/Hruschka/Pirbhai, "PSARE") added a *process* layer on top of the 1988 *notation*, plus refinements.

Process / Methods / Tools Precedence

The 2000 book's mantra: **process drives methods, methods drive tools** — pick in that order, not the reverse. Modern analog: "don't pick the tool first."

Concurrent Development Process

The 2000 book's alternative to waterfall: requirements, mechanisms, and architecture evolve *concurrently*, with iteration loops between them. Modern analog: agile-but-with-actual-architecture-work.

Superbubble

A **virtual bubble grouping multiple related bubbles** for navigation or abstraction purposes. Doesn't change the leveling rules; just a viewing aid. Modern analog: a "lens" or "selection" in a diagram tool.

Extended Traceability (added in 2000)

Refinement of 1988's traceability: links extend not just requirements→architecture but **requirements→mechanisms→architecture→implementation→tests**, with bidirectional navigation. See also: [Traceability](#).

Derived Requirement

A **requirement that emerges during architecture or design** — not in the original stakeholder ask, but necessary because of how the system is being built. The 2000 book makes these first-class so they're traceable too. Modern analog: "non-functional requirement discovered during implementation."

Object Orientation Integration

The 2000 book acknowledges UML had arrived and offers patterns for combining HP's structured analysis with OO design (class diagrams alongside DFDs, state charts in CSPECs). Modern analog: "structured analysis and OO are complementary, not exclusive."

Multiple Hierarchies (added in 2000)

Recognition that real systems live in **multiple overlapping hierarchies** (organizational, physical, functional, control). The 1988 book assumed one. The 2000 book lets you model several.

Meta-Model / Development Process Meta-Model

The 2000 book's framing of the *development process itself* using the same notation. The development project becomes a system, the team are processes, the deliverables are flows. Reflexive use of HP on the work of doing HP. Modern analog: "we used our own tool on our own project."

Business Object Architecture

A 2000-book pattern for organizing architecture around **business-domain objects** rather than purely-functional decomposition. Bridge between HP and domain-driven design.

Workflow-Centered Architecture

2000-book pattern: organize the architecture around **end-to-end workflows** (start at the user, follow the data, end at the persistence layer). Modern analog: vertical slice architecture.

9. Common Examples (from the 1988 book)

The HP books illustrate everything through four canonical worked examples, in increasing complexity. Worth knowing about as reference points:

- **Vending Machine** — transactional, discrete events, basic FSM. The "hello world" of HP.
 - **Home Heating System** — continuous control with sensors and actuators. Introduces continuous data flow and closed-loop control.
 - **Automobile Management** — multiple cooperating subsystems. Introduces inter-process control and decomposition across subsystems.
 - **Flight Management (Avionics)** — full real-time, safety-critical, redundant. The "go-to" complex example; the methodology's origin domain.
-

10. Antecedents & Influences

HP didn't appear in a vacuum — it builds on (and credits) several prior methods:

- **DeMarco Structured Analysis** (1979) — the original DFD notation, no control.
 - **Yourdon/Constantine Structured Design** (1979) — the architecture-side counterpart, structure charts.
 - **Ward-Mellor Real-Time Structured Analysis** (early 1980s) — an alternative real-time extension to DeMarco. HP and Ward-Mellor are parallel approaches with different control notations.
 - **Finite State Machine Theory** — the formal underpinning of CSPECs.
 - **Harel State Charts** (1987) — incorporated into the 2000 book.
 - **Putnam Model** — project sizing/scaling, referenced in the 1988 book.
-

This is a quick-reference card, not a tutorial. For the deep dive, see the source books: Hatley & Pirbhai (1988), Strategies for Real-Time System Specification; and Hatley, Hruschka & Pirbhai (2000), Process for System Architecture and Requirements Engineering.

Last updated: 2026-05-22.